

FACULTÉ DES SCIENCES DE BIZERTE

Département Informatique

Rapport de Stage

Stage d'été – Juillet 2026

Développement d'une application web de tirage au sort et de roue de la fortune

Réalisé par :
Trifi Yakine

Encadré par :
Mohammed Ali Ben Taher
Wael Gazzahi

Entreprise d'accueil : **AZIZA**
Année universitaire 2025 – 2026

Remerciements

Je tiens à exprimer mes sincères remerciements à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce stage.

Je remercie particulièrement Wael Gazzahi, mon encadrant au sein de l'entreprise AZIZA, pour sa disponibilité, ses conseils avisés et le temps qu'il m'a accordé tout au long de cette période.

Enfin, je remercie l'ensemble du personnel d'AZIZA pour leur accueil chaleureux, ainsi que les membres du jury pour l'intérêt qu'ils porteront à ce travail.

Résumé

Dans le cadre de ses campagnes promotionnelles, l'entreprise AZIZA organise régulièrement des tirages au sort destinés à sa clientèle. Ce rapport présente le travail réalisé durant un stage d'un mois consacré au développement d'une application web permettant d'automatiser la gestion de ces tirages au sort.

L'application développée permet l'import de listes de participants au format Excel ou CSV, la réalisation de tirages au sort via une roue de la fortune animée, la gestion d'un inventaire de cadeaux, la consultation de l'historique des gagnants ainsi qu'un tableau de bord statistique, le tout sécurisé par une authentification à deux facteurs (email et SMS). La solution s'appuie sur une architecture 3-tiers utilisant Angular pour le front-end, Laravel pour l'API REST back-end, et MySQL pour la persistance des données.

Mots-clés : tirage au sort, roue de la fortune, application web, Angular, Laravel, API REST, MySQL

Table des matières

Remerciements	i
Résumé	ii
Introduction générale	1
1 Analyse et spécification des besoins	2
Introduction	2
1.1 Identification de l'acteur	2
1.2 Besoins fonctionnels	2
1.3 Besoins non fonctionnels	3
1.4 Diagramme de cas d'utilisation	4
Conclusion	5
2 Conception	6
Introduction	6
2.1 Architecture de l'application	6
2.2 Conception de la base de données	6
Conclusion	7
3 Réalisation du back-end	8
Introduction	8
3.1 Environnement de travail	8
Environnement matériel	8
Environnement logiciel	8
3.2 Gestion des cadeaux	8
3.3 Gestion des participants	9
3.4 Logique du tirage au sort	10
3.5 Historique et export des gagnants	12
3.6 Tableau de bord	13
3.7 Authentification et sécurisation de l'API	13
Choix du canal d'envoi	13
Parcours d'authentification	15
3.8 Fonctionnalités complémentaires de l'API	15
Annulation d'un tirage	15

Journal des actions	15
Conclusion	16
4 Réalisation du front-end	17
Introduction	17
4.1 Architecture du projet Angular	17
4.2 Identité visuelle	17
4.3 Authentification côté client	18
Service et sécurisation des requêtes	18
Parcours utilisateur	18
4.4 Gestion des cadeaux	19
4.5 Import des participants	19
Renouvellement de campagne	19
Filtres et statut visuel	20
4.6 La roue de la fortune	20
Construction géométrique	20
Animation et synchronisation avec le tirage réel	20
Point technique	21
Transparence	21
Gestion des cas limites	21
4.7 Historique et export des gagnants	22
4.8 Tableau de bord	22
4.9 Gestion du compte administrateur	22
Changement de mot de passe	23
Création d'un administrateur	23
4.10 Fonctionnalités bonus de l'interface	23
Animations sonores	23
Affichage plein écran	24
Annulation et journal	24
Conclusion	24
Conclusion générale	25
Bibliographie et Webographie	27

Table des figures

1.1	Diagramme de cas d'utilisation de l'application	4
2.1	Architecture 3-tiers de l'application	6
2.2	Diagramme de classes de l'application	7

Liste des tableaux

1.1	Les besoins fonctionnels de l'application	3
-----	---	---

Introduction générale

La grande distribution connaît une transformation numérique profonde, qui ne se limite plus à la gestion des stocks ou à l'encaissement : les opérations marketing elles-mêmes – jeux-concours, tirages au sort, animations promotionnelles en magasin – gagnent à être outillées par des applications dédiées, garantes de fiabilité, de traçabilité et d'attractivité visuelle auprès du public.

C'est dans ce contexte que s'inscrit le présent stage, effectué au sein de l'enseigne tunisienne AZIZA. L'entreprise organise régulièrement des tirages au sort destinés à sa clientèle ; or, la gestion manuelle de ces opérations présente des limites bien identifiées : risque d'erreur dans la désignation des gagnants, absence de garantie qu'un même participant ne soit tiré qu'une seule fois, traçabilité insuffisante, et manque d'impact visuel lors d'événements organisés en direct.

L'objectif de ce stage est de concevoir et de développer une application web complète répondant à ces besoins : import de listes de participants, tirage au sort équitable matérialisé par une roue de la fortune animée, gestion des cadeaux mis en jeu, historique consultable et exportable des gagnants, tableau de bord de suivi, le tout sécurisé par une authentification à deux facteurs.

Ce rapport s'articule en quatre chapitres. Le premier est consacré à l'analyse et à la spécification des besoins. Le deuxième détaille la conception de la solution : architecture, base de données et API. Les troisième et quatrième chapitres décrivent respectivement la réalisation du back-end Laravel et celle du front-end Angular, en s'attardant sur les choix techniques et les difficultés rencontrées. Une conclusion générale dresse enfin le bilan du projet et ouvre sur ses perspectives d'évolution.

Chapitre 1

Analyse et spécification des besoins

Introduction

Ce chapitre est consacré à l'étude préliminaire du projet. Nous commençons par identifier l'acteur du système, puis nous dégageons les besoins fonctionnels et non fonctionnels auxquels l'application devra répondre. Enfin, nous modélisons ces besoins à l'aide du diagramme de cas d'utilisation.

1.1 Identification de l'acteur

Un acteur représente une entité externe qui interagit directement avec le système. L'étude du sujet fait ressortir un acteur unique :

- **L'administrateur** (également appelé animateur) : il est chargé de l'import et de la consultation des participants, de la gestion des cadeaux, du lancement des tirages au sort ainsi que de la consultation de l'historique des gagnants et du tableau de bord.

Il convient de souligner que le participant ne constitue pas un acteur du système. En effet, il n'interagit jamais directement avec l'application : il ne se connecte pas et n'effectue aucune action. Il est par conséquent représenté comme une entité manipulée par le système (une donnée), et non comme un acteur.

1.2 Besoins fonctionnels

Les besoins fonctionnels décrivent les services que le système doit rendre à son utilisateur. Le tableau 1.1 récapitule les besoins fonctionnels identifiés.

TABLE 1.1 – Les besoins fonctionnels de l’application

Code	Besoin fonctionnel	Description
BF1	Importer les participants	Importer une liste de participants à partir d’un fichier Excel (.xlsx) ou CSV, avec contrôle automatique des doublons.
BF2	Consulter les participants	Afficher la liste des participants ainsi que leur nombre total.
BF3	Gérer les cadeaux	Ajouter, modifier, supprimer, activer ou désactiver un cadeau et définir la quantité disponible.
BF4	Lancer un tirage au sort	Faire tourner la roue de la fortune qui désigne un cadeau, puis sélectionner aléatoirement un gagnant parmi les participants n’ayant jamais gagné.
BF5	Afficher le gagnant	Afficher le gagnant avec une animation : nom, prénom, téléphone, cadeau remporté, date et heure du tirage.
BF6	Consulter l’historique	Consulter l’historique complet des gagnants avec recherche, filtre par date et export au format Excel.
BF7	Consulter le tableau de bord	Visualiser les statistiques : nombre de participants, de gagnants, de participants restants et de cadeaux restants.
BF8	S’authentifier (bonus)	Permettre à l’administrateur de s’authentifier afin de sécuriser l’accès à l’application.

1.3 Besoins non fonctionnels

Les besoins non fonctionnels définissent les qualités et les contraintes que le système doit respecter, indépendamment des services rendus. Notre application doit garantir :

- **La fiabilité et l’équité** : le tirage doit être réellement aléatoire et l’unicité du gagnant doit être garantie, même en cas d’incident, grâce à l’utilisation de transactions au niveau de la base de données.
- **L’ergonomie** : l’interface doit être simple et intuitive, utilisable en direct devant un public lors d’un événement promotionnel.
- **La performance** : l’import de fichiers contenant plusieurs milliers de participants doit s’effectuer en quelques secondes et l’animation de la roue doit rester fluide.

- **La sécurité et la confidentialité** : les données personnelles des participants (notamment les numéros de téléphone) doivent être protégées et l'accès à l'administration doit être restreint.
- **La maintenabilité** : la séparation entre le front-end et le back-end à travers une API REST, ainsi qu'un code structuré et commenté, doivent faciliter l'évolution de l'application.
- **La portabilité** : l'application doit être compatible avec les navigateurs web récents (Chrome, Firefox, Edge).

1.4 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation modélise les interactions entre l'acteur et le système en décrivant les fonctionnalités offertes par l'application. La figure 1.1 illustre le diagramme de cas d'utilisation global de notre application.

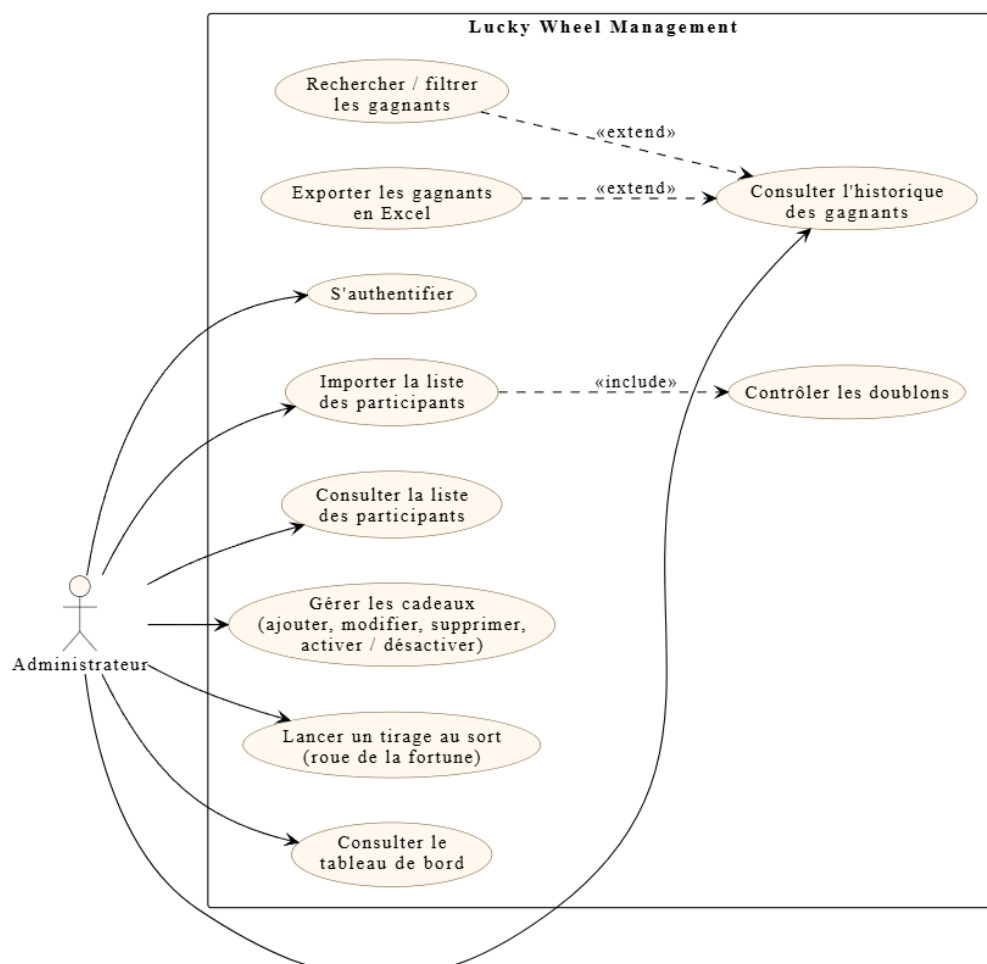


FIGURE 1.1 – Diagramme de cas d'utilisation de l'application

Comme le montre ce diagramme, l'administrateur accède à l'ensemble des fonctionnalités du système : l'import et la consultation des participants, la gestion des

cadeaux, le lancement des tirages, la consultation de l'historique des gagnants ainsi que du tableau de bord.

La relation « **include** » entre les cas « Importer la liste des participants » et « Contrôler les doublons » indique que le contrôle des doublons est systématiquement exécuté lors de chaque import : il s'agit d'un comportement obligatoire inclus dans le cas de base. En revanche, la relation « **extend** » entre « Exporter les gagnants en Excel » et « Consulter l'historique des gagnants » signifie que l'export est une fonctionnalité optionnelle, déclenchée uniquement à la demande de l'administrateur depuis l'écran de l'historique. De même, le cas « Rechercher / filtrer les gagnants » étend la consultation de l'historique : il n'est déclenché que lorsque l'administrateur souhaite retrouver un gagnant particulier.

Enfin, le cas d'utilisation « S'authentifier », prévu en bonus dans le cahier des charges, conditionne l'accès à l'ensemble des fonctionnalités de l'application afin d'en sécuriser l'utilisation.

Conclusion

Ce chapitre nous a permis de cerner le périmètre du projet à travers l'identification de l'acteur et la spécification des besoins fonctionnels et non fonctionnels, modélisés par le diagramme de cas d'utilisation. Le chapitre suivant sera consacré à la conception de la solution : architecture de l'application, modèle de la base de données et diagrammes de conception.

Chapitre 2

Conception

Introduction

Ce chapitre présente la conception technique de la solution retenue pour répondre aux besoins exprimés au chapitre précédent. Nous détaillons successivement l'architecture générale de l'application, la conception de la base de données, et la conception de l'API REST assurant la communication entre le front-end et le back-end.

2.1 Architecture de l'application

L'application repose sur une architecture 3-tiers, illustrée par la figure 2.1, qui sépare clairement les responsabilités entre trois couches indépendantes.

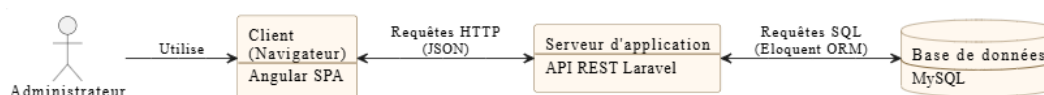


FIGURE 2.1 – Architecture 3-tiers de l'application

- La **couche cliente**, exécutée dans le navigateur, est développée avec le framework **Angular** et constitue une application monopage (SPA) ;
- La **couche applicative**, développée avec le framework **Laravel**, expose une API REST échangeant des données au format JSON ;
- La **couche de données**, assurée par **MySQL**, est interrogée par Laravel à travers son ORM Eloquent.

Cette séparation garantit l'indépendance des trois couches : le back-end pourrait, par exemple, être réutilisé par une future application mobile sans aucune modification.

2.2 Conception de la base de données

La base de données du système repose sur cinq entités principales, modélisées par le diagramme de classes de la figure 2.2.

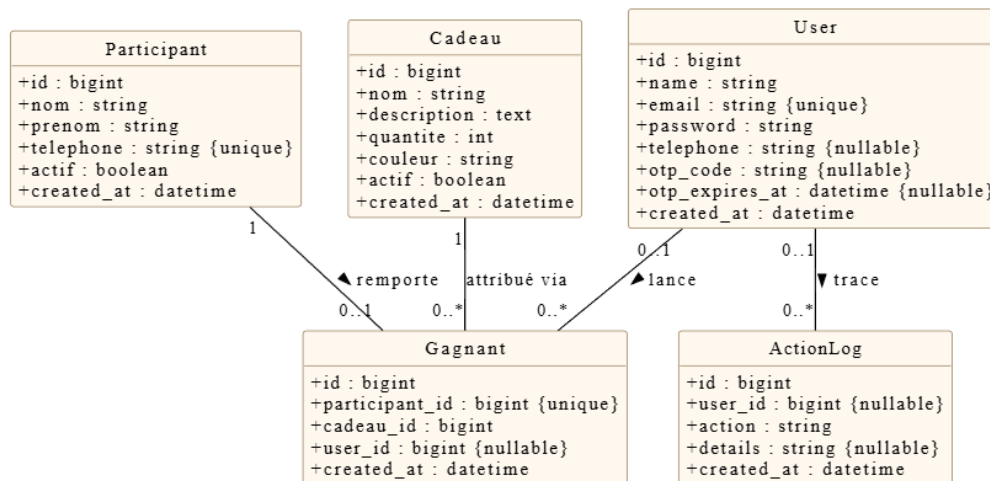


FIGURE 2.2 – Diagramme de classes de l'application

Conclusion

Ce chapitre a présenté les choix de conception retenus : une architecture 3-tiers séparant clairement front-end, back-end et base de données, un modèle de données garantissant l'intégrité des règles métier au niveau du système de gestion de base de données. Le chapitre suivant décrit la réalisation concrète de cette conception.

Chapitre 3

Réalisation du back-end

Introduction

Ce chapitre présente la mise en œuvre concrète de la conception décrite au chapitre précédent. Nous présentons d’abord l’environnement de travail et les technologies utilisées, puis nous détaillons, fonctionnalité après fonctionnalité, l’ensemble du back-end réalisé : gestion des cadeaux et des participants, logique du tirage au sort, historique des gagnants, tableau de bord, et enfin l’authentification sécurisée de l’application.

3.1 Environnement de travail

Environnement matériel

Environnement logiciel

- **Laragon** : environnement de développement local regroupant Apache, MySQL et PHP ;
- **Visual Studio Code** : éditeur de code ;
- **Postman** : outil de test des points d’entrée de l’API REST ;
- **HeidiSQL** : client graphique pour l’administration de la base de données MySQL ;
- **Git et GitHub** : gestion de versions et hébergement du dépôt du projet ;
- **PlantUML** : génération des diagrammes UML du projet.

3.2 Gestion des cadeaux

Un contrôleur `CadeauController` expose les cinq opérations CRUD standard (`index`, `store`, `show`, `update`, `destroy`) ainsi qu’une route dédiée à l’activation ou la désactivation d’un cadeau. Le Listing 3.1 illustre la gestion de la suppression, qui empêche la suppression d’un cadeau déjà attribué à un gagnant, contrainte garantie au niveau de la base de données par la clé étrangère `cadeau_id`.

```
1 public function destroy(Request $request, Cadeau $cadeau)
```

```
2 {
3     try {
4         $cadeau->delete();
5
6         ActionLog::ecrire('cadeau_supprime', $cadeau->nom, $request->
            user()?->id);
7
8         return response()->json(['message' => 'Cadeau supprime avec
                succes']);
9     } catch (\Illuminate\Database\QueryException $e) {
10         return response()->json([
11             'message' => 'Impossible de supprimer ce cadeau : il a
                deja ete attribue a un gagnant. Desactive-le plutot.',
12             ], 409);
13     }
14 }
```

Listing 3.1 – Suppression sécurisée d'un cadeau

Un point de conception notable concerne le routage : Laravel dérive automatiquement le nom du paramètre de route à partir du nom de la ressource, en appliquant des règles de pluralisation **anglaises**. Le mot **cadeaux** n'étant pas reconnu, ce paramètre était initialement généré de façon incohérente avec le nom attendu par le contrôleur (cadeaus), provoquant un échec silencieux de la liaison de modèle de route (*route model binding*) : le contrôleur recevait une instance vide plutôt que l'enregistrement réel. La correction consiste à déclarer explicitement le nom du paramètre :

```
1 Route::apiResource('cadeaux', CadeauController::class)
2     ->parameters(['cadeaux' => 'cadeau']);
```

Listing 3.2 – Déclaration explicite du paramètre de route

3.3 Gestion des participants

La gestion des participants repose sur deux fonctionnalités : la consultation de la liste existante et l'import en masse à partir d'un fichier Excel ou CSV (BF1). Plutôt que le package `maatwebsite/excel`, habituellement utilisé dans l'écosystème Laravel, nous avons opté pour un usage direct de la bibliothèque **PhpSpreadsheet**, qu'il enveloppe. Ce choix a été motivé par des incompatibilités documentées entre `maatwebsite/excel` et les versions récentes de Laravel et de PHP.

L'import s'appuie sur la classe `IOFactory` de `PhpSpreadsheet`, qui détecte automatiquement le format du fichier fourni. Trois catégories de lignes sont distinguées lors du traitement : les lignes importées avec succès, les doublons (détectés par comparaison du numéro de téléphone), et les lignes invalides (champs obligatoires manquants). Afin de

respecter l'exigence de performance du cahier des charges sur des fichiers de plusieurs milliers de lignes, l'ensemble des numéros de téléphone déjà présents en base est chargé en mémoire **avant** le parcours du fichier, évitant une requête SQL répétée à chaque ligne (problème dit du « N+1 »).

```
1 $telephonesExistants = Participant::pluck('telephone')-&gttoArray();
2
3 foreach ($lignes as $ligne) {
4     $donnee = array_combine($entetes, $ligne);
5     $telephone = trim((string) ($donnee['telephone'] ?? ''));
6     $nom = trim((string) ($donnee['nom'] ?? ''));
7     $prenom = trim((string) ($donnee['prenom'] ?? ''));
8
9     if (empty($telephone) || empty($nom) || empty($prenom)) {
10         $invalides++;
11         continue;
12     }
13
14     if (in_array($telephone, $telephonesExistants)) {
15         $doublons++;
16         continue;
17     }
18
19     try {
20         Participant::create(compact('nom', 'prenom', 'telephone'));
21         $telephonesExistants[] = $telephone;
22         $importes++;
23     } catch (\Illuminate\Database\QueryException $e) {
24         $doublons++;
25     }
26 }
```

Listing 3.3 – Traitement d'une ligne lors de l'import

Le bloc `try/catch` entourant la création constitue un filet de sécurité : même si la pré-vérification en mémoire laissait passer un doublon (par exemple en cas d'exécutions concurrentes), la contrainte `unique` posée sur la colonne `telephone` empêcherait l'insertion au niveau de la base de données.

3.4 Logique du tirage au sort

Cette section porte sur la fonctionnalité centrale de l'application (BF4), correspondant à l'endpoint `POST /api/tirages`. Le tirage doit satisfaire trois exigences simultanément : désigner un gagnant de façon réellement aléatoire, garantir qu'un participant ne puisse jamais gagner deux fois, et décrémenter le stock du cadeau concerné – le tout

de manière fiable même en cas d'erreur en cours d'exécution.

```
1 public function store(Request $request)
2 {
3     $request->validate(['cadeau_id' => 'required|exists:cadeaux,id']);
4
5     try {
6         $gagnant = DB::transaction(function () use ($request) {
7             $cadeau = Cadeau::findOrFail($request->cadeau_id);
8
9             if (!$cadeau->actif || $cadeau->quantite < 1) {
10                 throw new \RuntimeException('Ce cadeau n\'est plus
11                     disponible pour le tirage.');
```

```

44     } catch (\Illuminate\Database\QueryException $e) {
45         return response()->json(['message' => 'Une erreur est survenue
           , veuillez relancer le tirage.'], 409);
46     }
47 }

```

Listing 3.4 – Traitement d’un tirage au sort

Trois mécanismes garantissent la robustesse de cette opération :

latex

— **La sélection aléatoire du gagnant**

(`whereDoesntHave('gagnant')->inRandomOrder()->first()`) s’appuie sur la relation `hasOne` définie dans le modèle `Participant` – la traduction en code Eloquent de la multiplicité 1 vers 0..1 entre `Participant` et `Gagnant`, représentée dans le diagramme de classes du chapitre 2 : seuls les participants ne possédant **aucun** enregistrement associé dans la table `gagnants` sont éligibles. Le tirage aléatoire est délégué à MySQL plutôt qu’exécuté en PHP, ce qui est à la fois plus performant (aucun chargement de l’ensemble des participants en mémoire) et plus robuste.

— **La transaction** (`DB::transaction`) fait de l’ensemble de l’opération – vérification du cadeau, sélection du gagnant, création de l’enregistrement, décrémentation du stock – une unité indivisible : en cas d’échec à n’importe quelle étape, toutes les modifications sont annulées. Il ne peut donc jamais exister d’incohérence entre le stock et l’historique des gagnants.

— **Le décrémentation atomique** (`decrement('quantite')`) génère directement une instruction SQL de la forme `quantite = quantite - 1`, exécutée par le serveur de base de données lui-même, ce qui élimine tout risque de perte de mise à jour en cas d’opérations simultanées.

3.5 Historique et export des gagnants

L’historique des gagnants (BF6) est exposé par deux endpoints, `GET /api/gagnants` pour la consultation et `GET /api/gagnants/export` pour l’export au format Excel, tous deux s’appuyant sur une même méthode privée de construction de requête filtrée, afin d’éviter la duplication de la logique de recherche entre les deux.

Le filtre de recherche textuelle porte sur des colonnes de la table `participants` plutôt que `gagnants` ; il s’appuie donc sur `whereHas`, qu’Eloquent traduit en une sous-requête SQL :

```

1 $query->whereHas('participant', function ($q) use ($recherche) {
2     $q->where('nom', 'like', "%{$recherche}%")
3     ->orWhere('prenom', 'like', "%{$recherche}%")

```

```
4     ->orWhere('telephone', 'like', "%{$recherche}%");  
5 });
```

Listing 3.5 – Filtre de recherche sur l'historique des gagnants

L'export s'appuie de nouveau sur PhpSpreadsheet, cette fois en écriture : un classeur est généré dynamiquement à partir des résultats filtrés, puis renvoyé au client sous forme de fichier téléchargeable via `response()->download()`.

3.6 Tableau de bord

Le tableau de bord (BF7) expose un unique endpoint, `GET /api/dashboard`, retournant les statistiques globales de l'application : nombre total de participants, nombre de gagnants, participants restants, et cadeaux encore disponibles. Le nombre de participants restants est obtenu par simple soustraction plutôt que par une requête supplémentaire, ce qui n'est possible que grâce à la contrainte d'unicité posée sur `participant_id` dans la table `gagnants` : un gagnant correspond toujours à exactement un participant distinct.

3.7 Authentification et sécurisation de l'API

Au cours du stage, l'encadrant a formulé une exigence complémentaire par rapport au sujet initial : sécuriser l'accès à l'application par une authentification à deux facteurs (mot de passe puis code de vérification à usage unique), ainsi qu'un mécanisme de réinitialisation de mot de passe fonctionnant selon le même principe. Cette exigence prolonge le besoin BF8, initialement classé comme fonctionnalité bonus.

Choix du canal d'envoi

L'envoi des codes de vérification par SMS a été réalisé via **Twilio**, service tiers dont la compatibilité avec les numéros tunisiens a été vérifiée au préalable. Le compte d'essai gratuit, ne nécessitant aucune carte bancaire, fournit un crédit suffisant pour l'ensemble des besoins de test du projet. La logique d'envoi a été isolée dans une classe de service dédiée, séparée des contrôleurs :

```
1 class OtpService  
2 {  
3     private const DUREE_VALIDITE_MINUTES = 5;  
4  
5     public static function generateAndSend(User $user, string $canal =  
6         'email'): void  
7     {
```

```
7      $otp = random_int(100000, 999999);
8
9      $user->otp_code = $otp;
10     $user->otp_expires_at = now()->addMinutes(self::
        DUREE_VALIDITE_MINUTES);
11     $user->save();
12
13     if ($canal === 'sms') {
14         self::envoyerSms($user->telephone, $otp);
15     } else {
16         self::envoyerEmail($user->email, $otp);
17     }
18 }
19
20 private static function envoyerEmail(string $email, int $otp):
    void
21 {
22     $duree = self::DUREE_VALIDITE_MINUTES;
23     Mail::raw("Votre code de verification Lucky Wheel : {$otp} (
        valable {$duree} minutes)", function ($message) use (
        $email) {
24         $message->to($email)->subject('Code de verification -
            Lucky Wheel');
25     });
26 }
27
28 private static function envoyerSms(string $telephone, int $otp):
    void
29 {
30     $duree = self::DUREE_VALIDITE_MINUTES;
31     $client = new Client(config('services.twilio.sid'), config('
        services.twilio.token'));
32
33     $client->messages->create($telephone, [
34         'from' => config('services.twilio.from'),
35         'body' => "Lucky Wheel : votre code de verification est {
            $otp} (valable {$duree} minutes).",
36     ]);
37 }
38 }
```

Listing 3.6 – Service de génération et d'envoi des codes OTP

Parcours d'authentification

Deux parcours symétriques ont été implémentés, tous deux basés sur le même principe de vérification en deux temps :

- **Connexion** : `POST /api/login` (email et mot de passe) déclenche l'envoi d'un code, puis `POST /api/verify-otp` (email et code) délivre un jeton d'accès Sanctum en cas de correspondance.
- **Mot de passe oublié** : `POST /api/forgot-password` (email) déclenche l'envoi d'un code, puis `POST /api/reset-password` (email, code, nouveau mot de passe) applique le changement en cas de correspondance.

L'ensemble des routes métier de l'application (cadeaux, participants, tirages, gagnants, tableau de bord) a ensuite été placé derrière le middleware `auth:sanctum`, rendant le système d'authentification réellement contraignant plutôt que décoratif. Les routes d'authentification elles-mêmes sont limitées à six tentatives par minute (`throttle:6,1`) afin de prévenir tout usage abusif susceptible d'épuiser le crédit du service d'envoi de SMS.

3.8 Fonctionnalités complémentaires de l'API

Trois fonctionnalités bonus du cahier des charges ont été ajoutées en fin de projet, une fois le périmètre obligatoire couvert.

Annulation d'un tirage

En cas d'erreur (gagnant injoignable, tirage lancé par mégarde), l'administrateur peut annuler un tirage via l'endpoint `DELETE /api/gagnants/{id}`. L'opération, encapsulée dans une transaction, restaure le stock du cadeau concerné puis supprime l'enregistrement du gagnant – la disparition de la ligne levant du même coup la contrainte d'unicité, le participant redevient automatiquement éligible aux tirages suivants.

Journal des actions

La conception initiale du projet comportait une table de logs, écartée lors de la révision du diagramme de classes au motif que la colonne `user_id` de la table `gagnants` suffisait à tracer les tirages. Le périmètre du bonus étant plus large – il couvre également les *modifications* (cadeaux, imports) et les *annulations*, c'est-à-dire des événements qui ne laissent aucune trace dans les tables métier, voire qui en suppriment –, une table dédiée a finalement été réintroduite. Cette évolution illustre une distinction classique de conception : les tables *d'état* (`gagnants`, `cadeaux`) reflètent le présent et leurs lignes

peuvent disparaître, tandis qu’une table *de journal* est par nature en ajout seul (*append-only*) et ne doit jamais rien perdre. Une même table ne peut pas remplir les deux rôles.

La table `action_logs` comporte quatre colonnes utiles : `user_id` (clé étrangère nullable vers l’administrateur), `action` (identifiant du type d’événement), `details` (description libre) et `created_at`. Le modèle associé a été nommé `ActionLog` plutôt que `Log`, afin d’éviter toute collision avec la façade `Log` native de Laravel. Une méthode statique centralise l’écriture :

```
1 public static function ecrire(string $action, ?string $details = null,  
    ?int $userId = null): void  
2 {  
3     self::create([  
4         'action' => $action,  
5         'details' => $details,  
6         'user_id' => $userId,  
7     ]);  
8 }
```

Listing 3.7 – Méthode d’écriture du journal

Des appels à cette méthode ont été intégrés à chaque point de mutation de l’application : tirage effectué ou annulé, création, modification, suppression, activation ou désactivation d’un cadeau, et import de participants. L’endpoint `GET /api/logs` expose les deux cents entrées les plus récentes, accompagnées du nom de l’administrateur via la relation `belongsTo`.

Conclusion

Ce chapitre a détaillé l’ensemble des fonctionnalités back-end réalisées : gestion des cadeaux et des participants, logique du tirage au sort garantissant l’unicité des gagnants, historique consultable et exportable, tableau de bord, authentification sécurisée à deux facteurs, ainsi que les fonctionnalités complémentaires d’annulation, de journalisation et de statistiques. Le chapitre suivant sera consacré à la réalisation du front-end Angular, et notamment à l’interface de la roue de la fortune animée.

Chapitre 4

Réalisation du front-end

Introduction

Ce chapitre présente la réalisation de l'interface d'administration, développée avec Angular conformément à l'architecture définie au chapitre 2.

4.1 Architecture du projet Angular

Le projet a été initialisé avec Angular 22, en s'appuyant exclusivement sur des composants *standalone* (sans module Angular), conformément à l'orientation actuelle du framework. Le code est organisé en deux répertoires principaux sous `src/app/` :

- `core/` regroupe les éléments transverses à toute l'application : le service d'authentification, l'intercepteur HTTP chargé d'attacher le jeton d'accès à chaque requête, et le garde de route protégeant les pages nécessitant une connexion ;
- `features/` regroupe chaque page métier (cadeaux, participants, roue, historique, tableau de bord), chacune avec son propre composant et, le cas échéant, son service d'accès à l'API.

Chaque page communique avec le back-end Laravel via un service Angular dédié, exposant les données sous forme de *signals* plutôt que de propriétés de classe classiques – un choix cohérent avec le passage d'Angular à une stratégie de détection des changements `OnPush` par défaut, qui nécessite un état explicitement réactif pour que l'interface se mette à jour correctement.

4.2 Identité visuelle

Plutôt que de conserver une palette générique, l'interface reprend les couleurs de l'identité AZIZA (rouge et jaune), présentes sur l'ensemble des supports numériques de l'enseigne, notamment l'application AzizaPay. Un système de variables CSS centralise ces choix :

```
1 :root {  
2   --color-primary: #E4141E;
```

```
3  --color-accent: #FFC72C;  
4  --color-surface: #FFFFFF;  
5  --font-display: 'Baloo 2', sans-serif;  
6  --font-body: 'Inter', sans-serif;  
7  --radius-pill: 999px;  
8  }
```

Listing 4.1 – Extrait des variables de design partagées par toute l'application

Cette centralisation garantit une cohérence visuelle sur l'ensemble des pages sans dupliquer les mêmes valeurs dans chaque composant : le rouge porte l'identité de marque (actions principales, en-tête de navigation), tandis que le jaune reste un accent utilisé avec parcimonie (états actifs, mises en valeur), une combinaison choisie pour éviter la saturation visuelle propre à deux couleurs chaudes juxtaposées à parts égales.

4.3 Authentification côté client

Conformément au mécanisme mis en place côté back-end (chapitre 3), l'interface implémente un parcours de connexion en deux temps.

Service et sécurisation des requêtes

Un `AuthService` centralise l'état de connexion sous la forme d'un *signal* contenant le jeton d'accès, ainsi que les appels aux quatre endpoints d'authentification (connexion, vérification du code, mot de passe oublié, réinitialisation). Un intercepteur HTTP fonctionnel attache automatiquement ce jeton à chaque requête sortante :

```
1  export const authInterceptor: HttpInterceptorFn = (req, next) => {  
2    const token = inject(AuthService).token();  
3    if (token) {  
4      req = req.clone({ setHeaders: { Authorization: 'Bearer ${token}' }  
5        });  
6    }  
7    return next(req);  
8  };
```

Listing 4.2 – Intercepteur d'authentification

Un garde de route (`CanActivateFn`) redirige systématiquement vers l'écran de connexion tout accès à une page protégée en l'absence de jeton valide.

Parcours utilisateur

L'écran de connexion propose un choix explicite du canal de réception du code (email ou SMS), correspondant aux deux mécanismes implémentés côté serveur. Le

composant gère quatre états successifs pilotés par un *signal* (`password`, `otp`, `forgot-email`, `forgot-reset`), chacun affichant un formulaire distinct construit avec le module `ReactiveForms`, retenu pour sa validation déclarative et sa robustesse, plutôt que la nouvelle API *Signal Forms* introduite dans cette version d'Angular mais encore récente.

4.4 Gestion des cadeaux

La page de gestion des cadeaux reprend le CRUD exposé par l'API : une grille de fiches, chacune affichant une bande de couleur correspondant au champ `couleur` du cadeau (celui-là même qui définira la teinte du segment sur la roue), avec des actions rapides d'activation, de modification et de suppression. La création et la modification s'effectuent via une fenêtre modale unique, dont le titre et le comportement de soumission s'adaptent selon qu'un identifiant de cadeau est en cours d'édition ou non.

4.5 Import des participants

La page participants combine la consultation de la liste existante et l'import de nouveaux participants. L'envoi du fichier s'effectue via un objet `FormData` plutôt qu'un corps JSON classique :

```
1 importFile(file: File): Observable<ImportResult> {  
2   const formData = new FormData();  
3   formData.append('fichier', file);  
4   return this.http.post<ImportResult>(`${this.apiUrl}/import`,  
    formData);  
5 }
```

Listing 4.3 – Envoi du fichier d'import

Le résultat de l'import (nombre de lignes importées, doublons, lignes invalides), tel que renvoyé par l'API, est affiché sous forme de trois compteurs distincts et codés par couleur, offrant un retour immédiat et détaillé plutôt qu'un simple message de succès.

Renouvellement de campagne

Un besoin exprimé en cours de stage a conduit à faire de chaque import le point de départ d'une nouvelle campagne : les participants de l'import précédent n'ayant jamais gagné sont définitivement supprimés, tandis que les anciens gagnants sont conservés en base mais retirés de l'affichage (colonne `actif`, chapitre 3), pour préserver leur trace dans l'historique. Un quatrième compteur, ajouté au résumé d'import, indique le nombre de participants ainsi retirés. Une confirmation explicite est demandée avant tout import, cette opération étant partiellement destructive.

Filtres et statut visuel

Trois onglets (« Tous », « En attente de tirage », « Ont déjà gagné ») permettent de restreindre la liste affichée, à partir d'un champ calculé `a_gagne` ajouté à la réponse de l'API via `withExists()`, évitant une requête supplémentaire par participant. Chaque ligne du tableau est en outre mise en évidence par un badge de couleur reflétant ce statut.

4.6 La roue de la fortune

Cette section porte sur l'élément central de l'application, tant du point de vue technique que de la présentation du projet.

Construction géométrique

La roue est un unique élément SVG dont les segments sont calculés dynamiquement à partir de la liste des cadeaux actifs disposant d'un stock, plutôt que codés en dur. Pour un nombre de cadeaux donné, chaque segment occupe une portion égale du cercle (360° divisés par le nombre de cadeaux), et sa forme est décrite par un chemin SVG (*path*) calculé à partir de coordonnées polaires converties en coordonnées cartésiennes.

Animation et synchronisation avec le tirage réel

Un point de conception important distingue cette roue d'une simple animation décorative : **le cadeau désigné par la roue et le gagnant désigné par le serveur sont deux hasards indépendants**, qui doivent néanmoins converger vers une révélation unique et cohérente. Au lancement d'un tirage, un cadeau cible est choisi aléatoirement côté client pour déterminer la rotation finale de la roue, et ce même cadeau est transmis à l'endpoint `POST /api/tirages`, qui se charge du véritable tirage du participant. La révélation du résultat n'intervient que lorsque les deux conditions sont réunies : la fin de l'animation CSS *et* la réception de la réponse du serveur.

```
1 private tryReveal(): void {
2   if (!this.animationDone || (this.apiResult === null && this.apiError
    === null)) return;
3   this.spinning.set(false);
4   if (this.apiResult) {
5     this.genererConfettis();
6     this.winner.set(this.apiResult);
7   } else {
8     this.errorMessage.set(this.apiError);
9   }
10 }
```

Listing 4.4 – Attente conjointe de l’animation et de la réponse serveur

Ce mécanisme évite deux écueils symétriques : révéler un gagnant avant que la roue n’ait visuellement fini de tourner, ou inversement laisser la roue immobile alors que la réponse du serveur n’est pas encore arrivée. La rotation elle-même n’est jamais réinitialisée à zéro entre deux tirages : chaque nouveau tirage *ajoute* un nombre de degrés à la rotation déjà accumulée, garantissant que la roue tourne toujours vers l’avant, sans jamais revenir visuellement en arrière.

Point technique : orientation des étiquettes

Une première version orientait le texte de chaque segment radialement, en le faisant pivoter selon l’angle du segment concerné – une technique courante sur les graphiques circulaires. Cette approche s’est révélée fragile en pratique : une erreur de correction d’angle peut faire apparaître le texte inversé pour certains segments selon leur position, un défaut difficile à détecter systématiquement avant test visuel.

La solution retenue privilégie la robustesse à l’exhaustivité technique : chaque étiquette reste **toujours horizontale**, uniquement positionnée aux coordonnées du milieu de son segment, sans aucune rotation individuelle. Cette approche élimine par construction tout risque de texte inversé, quel que soit l’angle du segment ou l’état de rotation de la roue au repos. La taille de police s’ajuste par ailleurs dynamiquement au nombre de cadeaux actifs, afin de conserver une bonne lisibilité aussi bien avec trois qu’avec huit segments.

Transparence : panneau des participants éligibles

Un panneau repliable, affiché sous le bouton de tirage, liste les participants n’ayant pas encore gagné – utile pour rassurer un public assistant à un tirage en direct que celui-ci porte bien sur de réelles personnes. Seuls le prénom et le nom de chaque participant y figurent : le numéro de téléphone, donnée personnelle, reste exclu de tout affichage susceptible d’être projeté publiquement, y compris en mode plein écran.

Gestion des cas limites

Deux situations bloquent volontairement le lancement d’un tirage, chacune signalée par un message dédié au-dessus de la roue plutôt que par une simple erreur après clic : l’absence de cadeau actif disposant de stock, et l’absence de participant restant éligible. Dans ce second cas, la roue demeure visible (plutôt que remplacée par un message), seul le bouton de lancement étant désactivé – un choix d’interface délibéré pour que l’écran ne paraisse jamais « cassé ».

4.7 Historique et export des gagnants

La page d'historique reprend les capacités de filtrage exposées par l'API : recherche textuelle sur le participant et bornage par période, au moyen de deux champs de date encadrant un intervalle (`date_debut` et `date_fin`, chacun facultatif indépendamment de l'autre). L'export Excel s'appuie sur le même filtre actif à l'écran, afin que l'administrateur puisse exporter précisément la sélection qu'il consulte plutôt que l'intégralité de l'historique.

Le téléchargement du fichier binaire renvoyé par le serveur nécessite une prise en charge particulière côté Angular, la réponse HTTP ne pouvant être interprétée comme du JSON :

```
1 export(filtres): void {
2   this.http.get(`${this.apiUrl}/export`, {
3     params: this.buildParams(filtres),
4     responseType: 'blob',
5   }).subscribe((blob) => {
6     const url = window.URL.createObjectURL(blob);
7     const a = document.createElement('a');
8     a.href = url;
9     a.download = `gagnants-${new Date().toISOString().slice(0, 10)}.
      xlsx`;
10    a.click();
11    window.URL.revokeObjectURL(url);
12  });
13 }
```

Listing 4.5 – Téléchargement du fichier d'export

4.8 Tableau de bord

Le tableau de bord affiche les cinq statistiques exposées par l'API sous forme de cartes, chacune agissant comme un raccourci de navigation (`routerLink`) vers la page correspondante : les cartes liées aux participants renvoient vers la gestion des participants, celles liées aux cadeaux vers leur gestion, et celle des gagnants vers la page d'historique décrite ci-dessus.

4.9 Gestion du compte administrateur

Un menu accessible depuis l'avatar de l'administrateur, en permanence visible dans le bandeau de navigation, regroupe trois fonctions liées au compte plutôt que dispersées dans l'interface.

Changement de mot de passe

Un administrateur déjà connecté peut modifier son mot de passe en fournissant l'ancien puis le nouveau, sans passer par le circuit de vérification par code – à la différence du parcours « mot de passe oublié », destiné à un utilisateur qui ne peut précisément pas prouver son identité autrement. Les deux parcours partagent néanmoins une même règle de sécurité, appliquée côté serveur : le nouveau mot de passe doit différer du précédent, vérifié par comparaison de hachages (`Hash::check()`), le mot de passe stocké n'étant jamais accessible en clair.

Création d'un administrateur

Contrairement à un système d'inscription publique, la création d'un nouveau compte administrateur n'est accessible qu'à un administrateur déjà authentifié – un modèle de *provisionnement* plutôt que d'auto-inscription, cohérent avec un outil à usage strictement interne. Cette action, comme les autres, est tracée dans le journal des actions.

4.10 Fonctionnalités bonus de l'interface

Animations sonores

Le tirage est accompagné d'effets sonores générés par la **Web Audio API**, sans aucun fichier audio à héberger : chaque son est produit à la volée par un oscillateur paramétré en fréquence et en durée. Pendant la rotation, une séquence de « clics » est programmée avec des délais croissant quadratiquement, imitant le ralentissement mécanique d'une roue réelle ; une courte fanfare de quatre notes accompagne enfin la révélation du gagnant, en complément de l'animation de confettis.

```
1 playTickSequence(totalDurationMs: number): void {
2   const tickCount = 42;
3   let elapsed = 0;
4   for (let i = 0; i < tickCount; i++) {
5     const progress = i / tickCount;
6     const delay = 20 + progress * progress * 260;
7     elapsed += delay;
8     if (elapsed > totalDurationMs) break;
9     setTimeout(() => this.tick(), elapsed);
10  }
11 }
```

Listing 4.6 – Séquence de clics au rythme décroissant

Affichage plein écran

Pour une utilisation lors d'événements (roue projetée sur grand écran face au public), un bouton bascule la zone de la roue en plein écran via l'API native `requestFullscreen`. Le point notable est que cette API est appliquée à l'élément conteneur de la roue et non à la page entière : le bandeau de navigation, extérieur à cet élément, disparaît de lui-même, offrant un mode « vitrine » sans logique d'affichage supplémentaire. La pseudo-classe CSS `:fullscreen` agrandit la roue dans ce contexte, et un écouteur `fullscreenchange` maintient l'état de l'interface cohérent même lorsque l'utilisateur quitte le mode par la touche Échap.

Annulation d'un tirage et journal des actions

Côté interface, l'annulation d'un tirage (décrite au chapitre 4) est accessible depuis la page d'historique : chaque ligne comporte un bouton d'annulation, protégé par une demande de confirmation explicitant les conséquences (restauration du stock, retour du participant parmi les éligibles). Une nouvelle page « Journal » affiche par ailleurs les entrées du journal des actions, chaque type d'événement étant traduit en libellé lisible (« Tirage effectué », « Cadeau supprimé », « Import de participants »...), avec la date et le nom de l'administrateur concerné.

Conclusion

Ce chapitre a présenté la réalisation du front-end : architecture du projet Angular, identité visuelle reprenant les codes graphiques d'AZIZA, authentification à deux facteurs côté client, l'ensemble des pages de gestion prévues par le cahier des charges – cadeaux, participants, roue de la fortune, historique et tableau de bord – ainsi que les fonctionnalités bonus d'animation sonore, d'affichage plein écran, d'annulation de tirage et de journal des actions. L'ensemble des besoins fonctionnels, obligatoires comme optionnels, est désormais couvert par une application intégralement fonctionnelle, du back-end à l'interface d'administration.

Conclusion générale

Ce stage avait pour objectif le développement d'une application web de tirage au sort et de roue de la fortune destinée aux opérations marketing de l'enseigne AZIZA. Au terme du mois de travail, l'ensemble des besoins fonctionnels du cahier des charges est couvert par une application intégralement opérationnelle : import de participants avec détection des doublons, tirage au sort équitable garanti au niveau même de la base de données, roue animée synchronisée avec le tirage réel, gestion des cadeaux, historique consultable et exportable, et tableau de bord statistique.

Au-delà du périmètre obligatoire, les sept fonctionnalités bonus proposées par le sujet ont toutes été réalisées : authentification administrateur à deux facteurs (par email et par SMS), animations sonores lors du tirage, confettis à la révélation du gagnant, affichage plein écran pour les événements, annulation d'un tirage erroné, statistiques de campagne, et journal des actions. Plusieurs raffinements supplémentaires, non prévus au sujet initial, ont enfin été ajoutés à l'usage : un mécanisme de renouvellement de campagne à chaque import de participants, une gestion des comptes administrateurs (changement de mot de passe, provisionnement de nouveaux comptes), et un panneau de transparence affichant les participants éligibles pendant un tirage en direct.

Ce projet a été l'occasion de difficultés techniques formatrices, détaillées dans les chapitres de réalisation : les règles de pluralisation anglaises de Laravel appliquées à des noms français, le comportement silencieux de la protection contre le *mass assignment*, ou encore l'intégration de services externes d'envoi de SMS et d'emails. Chacune de ces difficultés a été résolue par une démarche de diagnostic méthodique, et documentée afin d'en conserver la trace.

Sur le plan des compétences, ce stage m'a permis de découvrir le framework Laravel et son ORM Eloquent, d'approfondir Angular dans ses évolutions récentes (composants autonomes, *signals*), et de mettre en pratique une organisation de projet en sprints avec gestion de versions systématique et rédaction progressive du présent rapport.

Ce rapport est accompagné d'un manuel utilisateur illustré, destiné à toute personne amenée à administrer l'application sans connaissance technique préalable, ainsi que d'une documentation technique à l'attention d'une éventuelle reprise du projet par une équipe de développement.

Plusieurs perspectives d'évolution peuvent enfin être envisagées : la formalisation d'une véritable notion de campagne en base de données (le mécanisme actuel de renouvellement à l'import en constituant une première approche fonctionnelle mais im-

plicite), la notification automatique des gagnants par SMS, une gestion des rôles différenciant les administrateurs, la conteneurisation de l'application avec Docker pour en faciliter le déploiement, ainsi que la mise en place de tests automatisés garantissant la non-régression lors des évolutions futures.

Bibliographie et Webographie

- Documentation officielle Laravel, <https://laravel.com/docs>
- Documentation officielle Angular, <https://angular.dev>
- Documentation officielle MySQL, <https://dev.mysql.com/doc/>